

Complete Technical Architecture & System Documentation

A Deep-Dive Architectural Blueprint, API Inventory, Database Schema, and Engineering Specification

Document Version	v2.4.0 (Enterprise Architecture Review)
System Name	DevFlow Workspace & Project Management Platform
Stack Classification	Full-Stack JavaScript (React 19, Node.js, Express, MongoDB, Socket.IO)
Target Audience	Engineering Leads, System Architects, Technical Interviewers, Developers
Primary Author / Engineer	Rudra Pratap Singh (DevFlow Core Team)
Repository Status	Production-Ready Verified Codebase
Publication Date	October 2026

Executive Summary: DevFlow is an enterprise-grade, full-stack collaborative project management platform engineered to mirror the workflow agility of modern platforms like Jira and Linear. It incorporates multi-tenant workspaces, scoped projects, interactive Kanban task boards with drag-and-drop state transitions, fine-grained Role-Based Access Control (Admin, Manager, Developer), real-time WebSockets notifications via Socket.IO, contextual comment threads, secure multipart file attachments, comprehensive activity audit logging, and dynamic analytics dashboards. This document provides an exhaustive, ground-truth analysis of every module, endpoint, schema, and security rule in the platform.

Table of Contents

1. Project Overview & Business Domain	Core objectives, target personas, problem statement, real-world utility
2. Complete Technology Stack Analysis	Frontend and backend libraries, architectural rationale, evaluation
3. High-Level System Architecture	Layered flow diagram, data flow, client-server protocol specifications
4. Folder Structure & Modular Organization	Clean separation of concerns, frontend and backend directories
5. Feature-by-Feature Deep Dive	16 implemented features: user flows, frontend, backend, security, interview pitches
6. Database Architecture & ER Design	8 Mongoose models, data dictionaries, schema constraints, relationship graph
7. Complete API Documentation & Inventory	Exhaustive documentation of all 44 endpoints with methods, roles, schemas
8. Complete End-to-End Project Workflow	Chronological user journey from registration through real-time collaboration
9. Security Architecture & Threat Mitigation	JWT lifecycle, bcrypt hashing, RBAC enforcement, query protection, CORS
10. Architectural Differentiators & USPs	Real-time sync, multi-workspace isolation, lightweight footprint
11. Scalability & Engineering Roadmap	Microservices evolution, caching layers, S3 cloud storage, AI copilots
12. Technical Interview Preparation Masterclass	2-min pitch, 5-min technical breakdown, engineering trade-offs

1. Project Overview & Business Domain

What is DevFlow? DevFlow is a modern, responsive web application engineered to solve the complex workflow and task coordination challenges encountered by software development teams, product managers, and engineering organizations. Inspired by leading industry tools such as Jira, Linear, and Trello, DevFlow combines hierarchical workspace management, interactive Kanban boards, real-time Socket.IO notifications, audit logging, and strict Role-Based Access Control (RBAC) into a high-performance, single-page application experience.

Core Problem Statement: In multi-tiered engineering teams, project visibility often degrades into fragmented communication channels (Slack, email, disconnected spreadsheets). Individual contributors lack centralized clarity on task priorities, managers struggle with cross-project tracking, and administrators face difficulties managing user privileges and team permissions. DevFlow resolves these pain points through a structured, multi-tenant hierarchy:

- **Workspaces** serve as top-level organizational containers representing departments, business units, or client spaces.
- **Projects** exist inside designated workspaces, scoping team members and sprint deliverables.
- **Tasks** reside within projects, encapsulating priorities, due dates, statuses, assignees, comment threads, and file attachments.
- **Audit Logs & Real-Time Sync** ensure instant situational awareness whenever a task is assigned, updated, or commented on.

Target User Persona	Primary Needs	DevFlow Solution
System Administrator	User governance, workspace lifecycle, role assignments, system health	Admin panel, direct user management, workspace CRUD, member enrollment
Project Manager / Lead	Sprint planning, project creation, task delegation, velocity tracking	Project management, Kanban task creation & assignment, status oversight
Software Developer	Task execution, status updates, technical discussions, specification docs	Personal task queues, Kanban column moves, comment discussions, file uploads

2. Complete Technology Stack Analysis

2.1 Frontend Technology Stack

Technology	Version	Architectural Rationale & Benefits	Application Usage
React	19.2.8	Component-driven rendering, fast reconciliation, custom hooks, and concurrent features for smooth UI transitions.	Entire UI layer: pages, forms, boards, modals, and context providers.
Vite	8.3.0	Next-generation lightning-fast build tool using native ES modules for sub-second HMR and optimized Rollup production bundling.	Dev server, build orchestrator, production asset packaging (`dist/`).
Tailwind CSS	3.4.19	Utility-first CSS framework enabling high-velocity UI development, custom design tokens, dark mode classes, and zero CSS runtime overhead.	All styling across layout, Kanban columns, badges, modals, and responsive breakpoints.
React Router	7.18.4	Client-side declarative routing with declarative route guards, URL search params synchronization, and history management.	`AppRoutes.jsx`, protected layout routes, dynamic `/tasks/:id` navigation.
Axios	1.20.0	Promise-based HTTP client featuring centralized interceptors for automatic JWT authorization header injection.	`src/services/api.js` instance communicating with all REST endpoints.
Socket.IO Client	4.8.4	Persistent WebSocket connection enabling instant server-to-client event streaming with automatic fallback and reconnection.	`NotificationContext.jsx` for live notifications and toast alerts.
React Context	Built-in	Lightweight, native state management eliminating external state overhead (Redux/Zustand) while guaranteeing predictable state flow.	`AuthContext`, `NotificationContext`, and `ThemeContext`.

2.2 Backend Technology Stack

Technology	Version	Architectural Rationale & Benefits	Application Usage
Node.js	v22.x	Event-driven, non-blocking asynchronous I/O engine ideal for real-time collaboration and concurrent API requests.	Server runtime executing the backend application using ES Modules.
Express.js	5.2.1	Minimalist, robust web framework providing robust middleware pipelines, route modularity, and error handling.	API routing (<code>/api/v1/*</code>), request parsing, security middleware, controllers.
MongoDB & Mongoose	9.10.1	Document-oriented NoSQL database with flexible JSON-like documents; Mongoose ODM enforces strict schemas and pre-save hooks.	Database persistence for Users, Workspaces, Projects, Tasks, Comments, etc.
JWT (jsonwebtoken)	9.0.3	Stateless, cryptographically signed tokens containing user identity and role payload for scalable authorization.	Access token (15m expiry) and Refresh token (7d expiry) generation and verification.
Bcryptjs	3.0.3	One-way cryptographic adaptive hashing with salt rounds to prevent rainbow table attacks.	User model pre-save password hashing and <code>comparePassword</code> method.
Socket.IO Server	4.8.4	Bi-directional, low-latency WebSocket communication engine featuring room-based event broadcasting.	Attached to HTTP server in <code>server.js</code> ; emits <code>notification:new</code> events.
Multer	2.4.0	Streaming multipart/form-data middleware with disk storage engines, file extension filters, and size constraints.	<code>attachment.controller.js</code> for handling file uploads (PDF, DOCX, PNG, JPG).
Security Suite	Helmet, CORS	Sets secure HTTP headers (XSS, HSTS, frameguard) and configures Cross-Origin Resource Sharing.	Mounted globally in <code>src/app.js</code> before all API routes.

3. Complete System Architecture

DevFlow implements a classic decoupled client-server architecture with an asynchronous event-driven real-time layer. The client communicates via two parallel channels: a RESTful HTTP channel for synchronous data mutations and queries, and a persistent WebSocket channel for live updates.



3.1 Request-Response Lifecycle Flow:

- Client Dispatch:** An interactive UI action (e.g. creating a project) calls an Axios helper in `src/services/api.js`.
- Interceptor Processing:** The Axios request interceptor extracts `accessToken` from `localStorage` and appends `Authorization: Bearer`.
- Security & Ingress:** Express parses cookies and JSON body; Helmet enforces security headers; CORS validates origin.
- Authentication Gate:** The `protect` middleware extracts the JWT, verifies cryptographic signature, checks user active status, and populates `req.user`.
- Authorization Filter:** The `authorize` middleware checks if `req.user.role` matches allowed roles (e.g. `[ 'admin', 'manager' ]`). If invalid, short-circuits with `403 Forbidden`.
- Controller Execution:** Controller executes validation, initiates Mongoose transactions, generates audit logs, and triggers real-time WebSocket events if applicable.
- Real-Time Fanout:** Socket.IO emits events directly to recipient rooms (`notification:new`), updating recipient browsers without requiring page reloads.
- JSON Response:** Standardized JSON envelope `{ success: true, message: '...', data: { ... } }` returned with appropriate HTTP status.

4. Folder Structure & Modular Organization

DevFlow follows an enterprise clean-architecture folder organization where concerns are strictly separated between the frontend client and the backend server. Neither layer relies on cross-boundary file imports, enabling independent CI/CD deployments and testing.

4.1 Frontend Directory Structure (`frontend/src`)

Directory	Responsibility & Architecture Role	Key Files
<code>src/components/</code>	Reusable, modular visual UI building blocks. Encapsulates presentation, micro-interactions, dropdowns, and guards.	<code>`Navbar.jsx`</code> , <code>`Sidebar.jsx`</code> , <code>`Layout.jsx`</code> , <code>`NotificationDropdown.jsx`</code> , <code>`ProtectedRoute.jsx`</code> , <code>`Toast.jsx`</code>
<code>src/pages/</code>	Top-level route views. Handles page-specific data fetching, modal states, form submissions, and orchestration.	<code>`Dashboard.jsx`</code> , <code>`Workspaces.jsx`</code> , <code>`Projects.jsx`</code> , <code>`TaskBoard.jsx`</code> , <code>`TaskDetails.jsx`</code> , <code>`Users.jsx`</code> , <code>`Login.jsx`</code>
<code>src/context/</code>	Global state providers using React Context API. Eliminates prop drilling for authentication, live notifications, and theming.	<code>`AuthContext.jsx`</code> , <code>`NotificationContext.jsx`</code> , <code>`ThemeContext.jsx`</code>
<code>src/routes/</code>	Route declaration registry integrating public routes and authenticated sub-routes wrapped in <code>`ProtectedRoute`</code> .	<code>`AppRoutes.jsx`</code>
<code>src/services/</code>	Networking layer configuring Axios client, baseURL, headers, and request interceptors.	<code>`api.js`</code>
<code>src/assets/</code>	Static graphics, SVG icons, and hero illustrations bundled into the production build.	<code>`hero.png`</code> , <code>`react.svg`</code> , <code>`vite.svg`</code>

4.2 Backend Directory Structure (`backend/src`)

Directory	Responsibility & Architecture Role	Key Files
src/controllers/	Business logic execution, parameter extraction, database operations, activity logging, and response dispatch.	`auth.controller.js`, `task.controller.js`, `project.controller.js`, `workspace.controller.js`, `user.controller.js`
src/routes/	REST route definitions mapping HTTP verbs and paths to middleware chains and controller handlers.	`index.js`, `auth.routes.js`, `workspace.routes.js`, `project.routes.js`, `task.routes.js`, `comment.routes.js`
src/models/	Mongoose schema definitions, field validations, indexes, references, and pre-save hooks.	`user.model.js`, `workspace.model.js`, `project.model.js`, `task.model.js`, `activityLog.model.js`
src/middleware/	Request interceptors for JWT validation (`protect`), role checks (`authorize`), and centralized 404/500 error handlers.	`auth.middleware.js`, `error.middleware.js`, `notFound.middleware.js`
src/sockets/	Socket.IO initialization, client connection authentication, user-room binding, and real-time notification dispatch.	`socket.js`
src/config/	External service configuration, environment variable validation, and MongoDB Atlas connection bootstrap.	`db.js`
src/utils/	Cryptographic helper utilities for access and refresh JWT token generation.	`generateTokens.js`

5. Feature-Wise Complete Architectural Analysis

5.1 Authentication System (Register, Login, JWT Tokens)

Purpose: Provide secure identity verification, cryptographic password storage, and stateless session management.

User Flow: 1. User submits name, email, password on `/register`. 2. Server creates account and redirects to `/login`. 3. User submits credentials on `/login`. 4. Server issues JWT Access & Refresh Tokens. 5. Client stores tokens and redirects to `/dashboard`.

Frontend Implementation: Pages: `Register.jsx`, `Login.jsx`. Context: `AuthContext.jsx` (`login`, `logout`, auto `/auth/me` verification). Storage: `localStorage.setItem('accessToken', token)` and `localStorage.setItem('user', JSON)`. UI shows loading spinners, error alerts, and redirect alerts.

Backend Implementation: Routes: `POST /api/v1/auth/register`, `POST /api/v1/auth/login`, `POST /api/v1/auth/refresh-token`, `POST /api/v1/auth/logout`, `GET /api/v1/auth/me`. Controller: `auth.controller.js`. Logic: Validates email regex and password length (≥ 6); hashes password via bcrypt pre-save; signs Access Token (15m) and Refresh Token (7d); sets HTTP-only cookies.

Database Impact: Model: `User` (`user.model.js`). Collection: `users`. Fields: `name`, `email` (unique index), `password` (hashed), `role`, `isActive`, `refreshToken`.

Security & Authorization: Bcrypt salt rounds (10), JWT signature verification with secret keys, deactivated account blocking (`isActive === false` returns 403), HTTP-only cookie support.

Architecture Impact: Establishes `req.user` identity context across all subsequent protected API endpoints in the Express request pipeline.

Technical Interview Explanation: *'We implemented a hybrid stateless JWT authentication system. Passwords are encrypted using bcrypt pre-save hooks. Upon login, an access token (15m) and refresh token (7d) are issued. An Axios interceptor automatically attaches the token to outgoing requests.'*

5.2 Role-Based Access Control (RBAC)

Purpose: Enforce principle of least privilege across Admin, Manager, and Developer personas.

User Flow: Users are assigned a role (`admin`, `manager`, or `developer`). When performing actions (e.g. creating a workspace), the server checks authorization. If restricted, a standardized 403 error is returned.

Frontend Implementation: Components conditionally render administrative controls based on `currentUser?.role`. E.g., `Sidebar.jsx` displays 'Users' link only for `admin`; `TaskBoard.jsx` renders assignee dropdown only for `admin` and `manager`.

Backend Implementation: Middleware: `authorize(...roles)` in `auth.middleware.js`. Enforced on routes: `authorize('admin')` on `/workspaces`, `authorize('admin', 'manager')` on `/projects` and `/tasks`. Controller: `updateTask` allows developers only to update status on assigned tasks.

Database Impact: Field: `role` in `User` model, with enum validation `['admin', 'manager', 'developer']`, default `developer`.

Security & Authorization: Enforced at the routing layer before controllers execute. Returns `{ success: false, message: 'You are not authorized to perform this action' }` with status 403.

Architecture Impact: Decouples authorization logic into reusable middleware functions, preventing duplicate permission checks inside business controllers.

Technical Interview Explanation: *'Our RBAC uses a higher-order middleware `authorize(...roles)` that inspects `req.user.role`. Admins possess full system governance, Managers can create projects and delegate tasks, while Developers are restricted to viewing assigned items and updating task progress.'*

5.3 Admin User Management

Purpose: Allow administrators to inspect registered accounts, view system roles, and trigger quick task assignments.

User Flow: 1. Admin navigates to `/users`. 2. Table renders all registered users with name, email, role, and active status. 3. Admin can search by keyword or click 'Assign Task' to jump directly to Kanban creation modal with user pre-selected.

Frontend Implementation: Page: `Users.jsx`. Protected by `isAdmin` check. Uses `API.get('/users?search=...')`. Renders user table with role badges, avatar initials, and quick navigation actions (`navigate('/task-board?assignTo=...&assignName;=...')`).

Backend Implementation: Route: `GET /api/v1/users`. Controller: `user.controller.js` (`getUsers`). Middleware: `protect`, `authorize('admin')`. Performs case-insensitive regex search on `name` and `email`.

Database Impact: Queries `User` collection. Returns sanitized projection excluding passwords and refresh tokens.

Security & Authorization: Strictly restricted to Admin role. Any non-admin request is intercepted by `authorize('admin')` returning 403.

Architecture Impact: Enables seamless user discovery for assignment workflows without requiring external directory integrations.

Technical Interview Explanation: *'The Users module is an Admin-only directory. It utilizes MongoDB regex queries for real-time user lookup and passes user IDs via query params to the TaskBoard modal for instantaneous pre-assigned task creation.'*

5.4 Workspace Management Hierarchy

Purpose: Provide multi-tenant organization containers grouping related projects, teams, and members.

User Flow: 1. Admin navigates to `/workspaces`. 2. Views existing workspaces or clicks 'Create Workspace'. 3. Enters workspace name and description. 4. Manages workspace members by enrolling users.

Frontend Implementation: Page: `Workspaces.jsx`. Features workspace selection cards, active workspace toggle, modal forms for workspace creation, and member lists.

Backend Implementation: Routes: `POST /api/v1/workspaces`, `GET /api/v1/workspaces`, `GET /api/v1/workspaces/:id`, `PUT /api/v1/workspaces/:id`, `DELETE /api/v1/workspaces/:id`, `POST /:id/members`, `DELETE /:id/members/:userId`. Controller: `workspace.controller.js`.

Database Impact: Model: `Workspace` (`workspace.model.js`). Collection: `workspaces`. Fields: `name`, `description`, `owner` (ref User), `members` (Array of ObjectId ref User).

Security & Authorization: Mutations (`POST`, `PUT`, `DELETE`, member enrollment) are locked to `admin`. Reads are scoped to authenticated members.

Architecture Impact: Root of the DevFlow entity hierarchy. Projects must reference a valid parent workspace.

Technical Interview Explanation: *'Workspaces act as the top-level boundary in our domain model. Only Admins can provision workspaces and enroll members. All projects and tasks cascade from their respective workspace.'*

5.5 Project Lifecycle Management

Purpose: Organize tasks within dedicated functional scopes, tracking progress milestones and team contributors.

User Flow: 1. User selects workspace. 2. Navigates to `/projects`. 3. Admin or Manager clicks 'Create Project'. 4. Fills project name, description, and status. 5. Project card displays progress metrics.

Frontend Implementation: Page: `Projects.jsx`. Displays project grids filtered by active workspace, search filtering, project creation modal, status badges (`Active`, `Completed`), and member counts.

Backend Implementation: Routes: `POST /api/v1/projects`, `GET /api/v1/projects`, `GET /:projectId`, `PUT /:projectId`, `DELETE /:projectId`, `POST /:projectId/members`, `DELETE /:projectId/members/:userId`, `GET /:projectId/activity`. Controller: `project.controller.js`.

Database Impact: Model: `Project` (`project.model.js`). Collection: `projects`. Fields: `name`, `description`, `workspace` (ref Workspace), `owner` (ref User), `members` (ref User array), `status`.

Security & Authorization: Creation and modification restricted to `admin` and `manager`. Only enrolled members can access project tasks and activity.

Architecture Impact: Direct parent container for tasks and project-level activity logs.

Technical Interview Explanation: *'Projects bridge high-level workspaces with granular tasks. Admins and Managers manage project scopes, while developers collaborate within assigned projects. Activity logs automatically capture project lifecycle events.'*

5.6 Kanban Task Board

Purpose: Offer an agile, visual workflow board for tracking task statuses across Todo, In Progress, and Done.

User Flow: 1. User opens `/task-board`. 2. Selects active project. 3. Tasks are organized into three visual status columns. 4. Users can change task status, trigger search, filter by priority or assignee, or open detailed view.

Frontend Implementation: Page: `TaskBoard.jsx`. Renders 3 status columns (`Todo`, `In Progress`, `Done`). Includes filter toolbar (Search input, Priority selector, Assignee selector). Status updates execute optimistic local state updates before API response.

Backend Implementation: Routes: `GET /api/v1/tasks?project=...&status;=...&priority;=...&assignedTo;=...&search;=...`, `PUT /api/v1/tasks/:id` with `{ status }`. Controller: `task.controller.js`.

Database Impact: Model: `Task` (`task.model.js`). Collection: `tasks`. Fields: `title`, `description`, `project`, `workspace`, `assignedTo`, `createdBy`, `status`, `priority`, `dueDate`.

Security & Authorization: Scoped by project membership. Developers can update status of their own assigned tasks.

Architecture Impact: Primary operational interface for daily developer productivity and sprint tracking.

Technical Interview Explanation: *'Our Kanban board groups tasks by status with multi-criteria filtering. Developers update their assigned task statuses in real-time, triggering automatic Socket.IO notifications to project collaborators.'*

5.7 Task Assignment System

Purpose: Enable leadership to assign tasks to registered users with role and project validation.

User Flow: 1. Admin/Manager opens Create Task modal or Task Details. 2. Assignee dropdown displays all registered members in `Name (Role)` format (e.g. `Rahul Sharma (Developer)`). 3. Upon selection, task is assigned and real-time notification is dispatched.

Frontend Implementation: Component: `TaskBoard.jsx` modal and `TaskDetails.jsx`. Fetches available users via `GET /users` for Admin/Manager. Pre-selects assignee if query parameters exist. Drops assigned user ID into `assignedTo` payload.

Backend Implementation: Routes: ``POST /api/v1/tasks``, ``PUT /api/v1/tasks/:id``. Controller: ``task.controller.js``. Logic: Validates assignee exists in MongoDB, verifies project membership, creates ``TASK_ASSIGNED`` notification in DB, and emits Socket.IO event.

Database Impact: Field: ``assignedTo`` (ObjectId ref User). Populated on query: ``populate('assignedTo', 'name email avatar')``.

Security & Authorization: Developers are strictly blocked from reassigning tasks to others. Only Admins, Managers, and Project Owners can assign/reassign.

Architecture Impact: Direct integration point between User Directory, Task Board, and Notification Center.

Technical Interview Explanation: *'Task assignment accepts user IDs and enforces project membership checks. When a task is assigned, the backend persists a notification document and immediately fires a WebSocket event to the assignee's dedicated room.'*

5.8 Contextual Commenting System

Purpose: Provide threaded task discussion streams allowing team members to communicate without leaving the task context.

User Flow: 1. User opens task details at ``/tasks/:id``. 2. Views discussion thread. 3. Enters comment in input box. 4. Submits comment. 5. Comment appears with author name, timestamp, and avatar. Author can edit or delete comment.

Frontend Implementation: Page: ``TaskDetails.jsx``. Renders comment list, input form, edit/delete actions, relative timestamp formatting, and author validation.

Backend Implementation: Routes: ``POST /api/v1/tasks/:taskId/comments``, ``GET /api/v1/tasks/:taskId/comments``, ``PUT /api/v1/comments/:id``, ``DELETE /api/v1/comments/:id``. Controller: ``comment.controller.js``. Logic: Saves comment, populates user avatar, generates ``COMMENT_ADDED`` notifications for project collaborators.

Database Impact: Model: ``Comment`` (``comment.model.js``). Collection: ``comments``. Fields: ``task`` (ref Task), ``user`` (ref User), ``text``.

Security & Authorization: Protected route (``protect``). Comment update/delete restricted to comment author or project owner.

Architecture Impact: Eliminates external communication silos by keeping technical context attached directly to the task document.

Technical Interview Explanation: *'Comments provide in-place discussions on tasks. Authors can update or delete their comments, and posting a comment triggers real-time notification alerts to all relevant task participants.'*

5.9 File Attachment System

Purpose: Enable users to upload specification documents, screenshots, and assets directly to tasks.

User Flow: 1. User opens ``/tasks/:id``. 2. Clicks file upload input. 3. Selects document (PDF, DOCX, PNG, JPG). 4. File uploads with progress indicator. 5. Attachment appears with file size, type badge, and download/delete buttons.

Frontend Implementation: Page: ``TaskDetails.jsx``. Uses ``FormData`` API to send multipart payload. Displays file cards with size formatting (``KB/MB``), extension icons, direct download links, and delete options.

Backend Implementation: Routes: ``POST /api/v1/tasks/:taskId/attachments``, ``GET /api/v1/tasks/:taskId/attachments``, ``DELETE /api/v1/attachments/:id``. Controller: ``attachment.controller.js``. Middleware: Multer disk storage in ``uploads/``, 5MB limit, whitelist filter (``pdf``, ``docx``, ``png``, ``jpg``, ``jpeg``).

Database Impact: Model: ``Attachment`` (``attachment.model.js``). Collection: ``attachments``. Fields: ``task``, ``filename``, ``originalName``, ``filePath``, ``mimeType``, ``size``, ``uploadedBy``.

Security & Authorization: MIME type and extension whitelisting prevent malicious file execution. Deletion permitted only by uploader or project owner.

Architecture Impact: Enhances task specification accuracy with direct asset linking.

Technical Interview Explanation: *'Our attachment subsystem uses Multer with disk storage and strict file-type whitelisting (PDF, DOCX, PNG, JPG) up to 5MB. Files are served statically and tracked via Mongoose attachment documents.'*

5.10 Real-Time Socket.IO Notification Center

Purpose: Deliver instant real-time alerts when tasks are assigned, statuses change, or comments are added.

User Flow: 1. User logs in. 2. Navbar bell icon displays unread badge counter. 3. When an event occurs (e.g. assigned a task), an ephemeral toast appears instantly and unread counter increments. 4. User clicks bell to inspect notifications and mark as read.

Frontend Implementation: Component: ``NotificationDropdown.jsx``, ``Toast.jsx``. Context: ``NotificationContext.jsx``. Connects to Socket.IO using JWT authentication. Listens for ``notification:new``, displays toast, updates unread count, calls ``PATCH /notifications/:id/read``.

Backend Implementation: Server: ``server.js`` (HTTP server wrapper). Sockets: ``socket.js``. Authenticates socket handshake via JWT, joins user to room ``user._id``. Function ``emitNotification(userId, notification)`` emits to room.

Database Impact: Model: ``Notification`` (``notification.model.js``). Collection: ``notifications``. Fields: ``recipient`` (ref User), ``sender`` (ref User), ``type``, ``message``, ``task``, ``isRead``, ``readAt``.

Security & Authorization: Socket handshake authenticated using ``token``. Notifications scoped strictly to the recipient's private room.

Architecture Impact: Transforms static polling into a reactive event-driven push architecture.

Technical Interview Explanation: *'We built a full real-time notification engine with Socket.IO. When controllers log critical actions, they persist a notification in MongoDB and trigger `emitNotification`, which sends the event directly to the recipient's room over WebSockets.'*

5.11 Activity Timeline & Project Audit Logging

Purpose: Maintain an immutable, chronological audit trail of all project modifications for accountability.

User Flow: 1. User opens ``/activity`` or navigates to project activity tab. 2. Timeline displays chronological list of actions (e.g. 'Jane Doe created project Alpha', 'John assigned task #12 to Bob').

Frontend Implementation: Page: ``ActivityTimeline.jsx``. Formats activity stream with action icons, humanized relative timestamps ('2 hours ago'), actor name badges, and project links.

Backend Implementation: Routes: ``GET /api/v1/projects/:projectId/activity``, ``GET /api/v1/dashboard/recent-activities``. Controllers: ``project.controller.js``, ``task.controller.js`` call ``ActivityLog.create()`` upon major mutations.

Database Impact: Model: ``ActivityLog`` (``activityLog.model.js``). Collection: ``activitylogs``. Fields: ``user`` (ref User), ``project`` (ref Project), ``action`` (e.g. ``PROJECT_CREATED``, ``TASK_STATUS_UPDATED``), ``description``.

Security & Authorization: Read-only for project members. Audit records cannot be modified or deleted via API.

Architecture Impact: Provides compliance, observability, and debugging history for team collaboration.

Technical Interview Explanation: *'Activity logging acts as our audit ledger. Controllers write immutable ActivityLog documents during task and project operations, which the frontend renders as an interactive chronological timeline.'*

5.12 Dashboard Analytics & Metrics Aggregation

Purpose: Provide executive summary cards, task distribution metrics, and quick personal task queues.

User Flow: 1. User lands on `/dashboard` after login. 2. Views 4 KPI cards (Total Projects, Total Tasks, Completed Tasks, Active Workspaces). 3. Inspects Task Status breakdown bar (Todo vs In Progress vs Done). 4. Views Priority breakdown (High, Medium, Low) and personal tasks.

Frontend Implementation: Page: `Dashboard.jsx`. Executes parallel API calls (`getOverview`, `getTaskStatus`, `getTaskPriority`, `getMyTasks`). Renders styled KPI cards, percentage progress bars, priority badges, and quick-action buttons.

Backend Implementation: Routes: `GET /api/v1/dashboard/overview`, `/task-status`, `/task-priority`, `/recent-tasks`, `/recent-activities`, `/my-tasks`. Controller: `dashboard.controller.js`. Logic: Uses MongoDB `countDocuments` and aggregation pipelines scoped to user's projects.

Database Impact: Aggregates across `Task`, `Project`, and `Workspace` collections using indexed status and membership queries.

Security & Authorization: Scoped strictly to projects where `members: req.user.\_id`.

Architecture Impact: High-level entry point delivering immediate visibility into sprint velocity and personal priorities.

Technical Interview Explanation: *'The dashboard aggregates team metrics through parallel asynchronous queries. It calculates task completion percentages, priority distributions, and filters personal queues based on the logged-in user.'*

6. Database Architecture & ER Design

DevFlow utilizes MongoDB Atlas paired with Mongoose ODM. Schemas enforce type validation, default values, foreign key references via ObjectIds (ref), unique compound indexes, and pre-save lifecycle hooks. The relational hierarchy follows a clean cascaded model:

```
+-----+ 1 : N +-----+ 1 : N +-----+ | USER | -----> |
WORKSPACE | -----> | PROJECT | |-----| |-----| |-----|
| _id (ObjectId) | | _id (ObjectId) | | _id (ObjectId) | | name (String) | | name (String) | | name (String) |
| email (Unique) | | description (Str) | | workspace (RefWS)| | password (Hash) | | owner (Ref User) | | owner
(Ref User) | | role (Enum) | | members ([RefUser]) | members ([RefUser) | isActive (Bool) | | createdAt, ... |
| status (Enum) | +-----+ +-----+ +-----+ | | 1 : N | (AssignedTo,
CreatedBy) v | +-----+
+-----> | TASK | |-----| |
| _id (ObjectId) | project (Ref Project) | | title (String) | workspace (Ref Workspace) | | description (String)
| assignedTo (Ref User, Nullable) | | status (Todo/Progress/Done)| createdBy (Ref User) | | priority
(Low/Med/High) | dueDate (Date, Nullable) | +-----+
| | | 1 : N | 1 : N | 1 : N v v v +-----+ +-----+ +-----+ | COMMENT |
| ATTACHMENT | | NOTIFICATION | |-----| |-----| |-----| | _id
(ObjectId) | | _id (ObjectId) | | _id (ObjectId) | | task (Ref Task) | | task (Ref Task) | | recipient (Ref) |
| user (Ref User) | | filename (String)| | sender (Ref) | | text (String) | | filePath (String)| | type (Enum)
| | createdAt, ... | | size, mimeType | | isRead, readAt | +-----+ +-----+
+-----+
```

6.1 Data Dictionary & Schema Specifications:

Model: User (Collection: users)

Represents authenticated user accounts and privileges.

Field Name	Type & Constraints	Description
name	String (Required, Trim)	User's full display name
email	String (Required, Unique, Lowercase)	Authentication identifier with email regex validation
password	String (Required, Hashed)	Bcrypt hashed password string (never stored in plain text)
role	String (Enum: admin, manager, developer)	Access control classification, defaults to 'developer'
avatar	String	Profile picture URL or empty string
isActive	Boolean (Default: true)	Account status; inactive accounts blocked by auth middleware
refreshToken	String (Nullable)	Current valid refresh token hash

Model: Workspace (Collection: workspaces)

Multi-tenant container grouping projects and teams.

Field Name	Type & Constraints	Description
name	String (Required, Trim)	Unique workspace name
description	String	Optional descriptive context
owner	ObjectId -> User (Required)	Creator/Admin who manages the workspace
members	[ObjectId -> User]	Array of enrolled user IDs with access rights

Model: Project (Collection: projects)

Functional deliverable scope inside a workspace.

Field Name	Type & Constraints	Description
name	String (Required, Trim)	Project title
description	String	Project charter or brief
workspace	ObjectId -> Workspace (Required)	Parent workspace reference
owner	ObjectId -> User (Required)	Project manager or creator
members	[ObjectId -> User]	Team members authorized to view and interact with tasks
status	String (Enum: Active, Completed)	Project lifecycle state, defaults to 'Active'

Model: Task (Collection: tasks)

Granular actionable unit with status, priority, and assignees.

Field Name	Type & Constraints	Description
title	String (Required, Trim)	Task summary title
description	String	Detailed acceptance criteria / markdown
project	ObjectId -> Project (Required)	Parent project reference
workspace	ObjectId -> Workspace	Parent workspace reference
assignedTo	ObjectId -> User (Nullable)	Assigned developer ID
createdBy	ObjectId -> User (Required)	Task creator ID
status	String (Enum: Todo, In Progress, Done)	Current workflow column, defaults to 'Todo'
priority	String (Enum: Low, Medium, High)	Urgency level, defaults to 'Medium'
dueDate	Date (Nullable)	Deadline for task completion

Model: Comment (Collection: comments)

Threaded discussion entry attached to a task.

Field Name	Type & Constraints	Description
task	ObjectId -> Task (Required)	Parent task reference
user	ObjectId -> User (Required)	Comment author
text	String (Required, Trim)	Comment content

Model: Attachment (Collection: attachments)

Uploaded asset or file linked to a task.

Field Name	Type & Constraints	Description
task	ObjectId -> Task (Required)	Parent task reference
filename	String (Required)	Unique disk filename generated by Multer
originalName	String (Required)	Original filename submitted by user
filePath	String (Required)	Relative server storage path ('uploads/...')
mimeType	String (Required)	Validated MIME type
size	Number (Required)	File size in bytes (max 5MB)

uploadedBy	ObjectId -> User (Required)	Uploader reference
------------	-----------------------------	--------------------

Model: Notification (Collection: notifications)

In-app and real-time alert document.

Field Name	Type & Constraints	Description
recipient	ObjectId -> User (Required)	Target user receiving alert
sender	ObjectId -> User (Required)	Actor triggering notification
type	String (Enum: TASK_ASSIGNED, TASK_STATUS_CHANGED, COMMENT_ADDED)	Alert category
message	String (Required)	Human-readable notification text
task	ObjectId -> Task	Related task reference
isRead	Boolean (Default: false)	Read/unread indicator
readAt	Date (Nullable)	Timestamp when marked as read

Model: ActivityLog (Collection: activitylogs)

Audit trail entry recording system events.

Field Name	Type & Constraints	Description
user	ObjectId -> User (Required)	Actor responsible for mutation
project	ObjectId -> Project (Required)	Project context
action	String (Required)	Action key (e.g. PROJECT_CREATED, TASK_ASSIGNED)
description	String (Required)	Detailed audit log statement

7. Complete API Documentation & Inventory

This section documents the complete inventory of all **44 REST API endpoints** implemented in the DevFlow backend. Every endpoint is cataloged with its exact HTTP method, URI path, authentication requirement, RBAC permissions, controller mapping, and functional purpose.

Total Implemented APIs 44	Protected Endpoints 39	Public Endpoints 5	Admin-Restricted 7	Admin & Manager 7
------------------------------	---------------------------	-----------------------	-----------------------	----------------------

Module	Total APIs	Auth Requirements	Functional Scope
Authentication	7	Public / Protected	Signup, login, logout, token refresh, `/me` profile
Users Management	1	Admin Only	Registered user directory and search
Workspaces	7	Admin / Members	Workspace CRUD, member addition and removal
Projects	8	Admin, Manager / Members	Project CRUD, members, project activity logs
Tasks	5	Admin, Manager / Developer	Task creation, board list, update, deletion
Comments	4	Protected (All Members)	Task discussion thread, edit, delete
Attachments	3	Protected (All Members)	Multipart file upload, listing, deletion
Dashboard Analytics	6	Protected (All Members)	KPI overview, status bar, priority, personal tasks
Notifications	3	Protected (Recipient)	Fetch alerts, mark single read, mark all read

7.1 Exhaustive API Endpoint Inventory:

Method & Endpoint	Auth Level	Handler / Controller	Purpose & UI Component
GET <code>/api/v1/auth</code>	Public None	auth.controller.js Model: None	Health check for auth routing module. UI: Home / Diagnostics
POST <code>/api/v1/auth/register</code>	Public None	register Model: User	Register new user with hashed password. UI: Register.jsx
POST <code>/api/v1/auth/login</code>	Public None	login Model: User	Authenticate credentials; returns JWT access/refresh tokens. UI: Login.jsx
POST <code>/api/v1/auth/refresh-token</code>	Public None	refreshTokenHandler Model: User	Refresh expired access token using refresh token. UI: api.js interceptor
POST <code>/api/v1/auth/logout</code>	Public None	logout Model: User	Clear refresh token in DB and expire HTTP cookies. UI: Navbar.jsx (Logout)
GET <code>/api/v1/auth/me</code>	Protected protect	getMe Model: User	Fetch current authenticated user profile. UI: AuthContext.jsx
GET <code>/api/v1/auth/admin-only</code>	Protected protect, authorize('admin')	inline handler Model: User	Verify admin permission gate. UI: Verification / Tests
GET <code>/api/v1/users</code>	Protected protect, authorize('admin')	getUsers Model: User	List registered users with regex search. UI: Users.jsx , TaskBoard.jsx
POST <code>/api/v1/workspaces</code>	Protected protect, authorize('admin')	createWorkspace Model: Workspace	Create new organizational workspace. UI: Workspaces.jsx
GET <code>/api/v1/workspaces</code>	Protected protect	getWorkspaces Model: Workspace	Get all workspaces where user is enrolled. UI: Workspaces.jsx , Navbar.jsx
GET <code>/api/v1/workspaces/:id</code>	Protected protect	getWorkspaceById Model: Workspace	Get details of single workspace by ID. UI: Workspaces.jsx
PUT <code>/api/v1/workspaces/:id</code>	Protected protect, authorize('admin')	updateWorkspace Model: Workspace	Update workspace name and description. UI: Workspaces.jsx
DELETE <code>/api/v1/workspaces/:id</code>	Protected protect, authorize('admin')	deleteWorkspace Model: Workspace	Permanently delete workspace and children. UI: Workspaces.jsx
POST <code>/api/v1/workspaces/:id/members</code>	Protected protect, authorize('admin')	addMember Model: Workspace	Enroll user into workspace membership. UI: Workspaces.jsx
DELETE <code>/api/v1/workspaces/:id/members/:userId</code>	Protected protect, authorize('admin')	removeMember Model: Workspace	Remove member from workspace. UI: Workspaces.jsx
POST <code>/api/v1/projects</code>	Protected protect, authorize('admin', 'manager')	createProject Model: Project	Create new project inside workspace. UI: Projects.jsx
GET <code>/api/v1/projects</code>	Protected protect	getProjects Model: Project	Get projects for user with optional workspace filter. UI: Projects.jsx , TaskBoard.jsx

GET <code>/api/v1/projects/:projectId</code>	Protected protect	getProjectById Model: Project	Get single project by ID with members. UI: Projects.jsx
PUT <code>/api/v1/projects/:projectId</code>	Protected protect, authorize('admin', 'manager')	updateProject Model: Project	Update project name, description, status. UI: Projects.jsx
DELETE <code>/api/v1/projects/:projectId</code>	Protected protect, authorize('admin', 'manager')	deleteProject Model: Project	Delete project and associated tasks. UI: Projects.jsx
POST <code>/api/v1/projects/:projectId/members</code>	Protected protect, authorize('admin', 'manager')	addProjectMember Model: Project	Add member to project. UI: Projects.jsx
DELETE <code>/api/v1/projects/:projectId/members/:userId</code>	Protected protect, authorize('admin', 'manager')	removeProjectMember Model: Project	Remove member from project. UI: Projects.jsx
GET <code>/api/v1/projects/:projectId/activity</code>	Protected protect	getProjectActivityLogs Model: ActivityLog	Get activity audit logs for project. UI: ActivityTimeline.jsx
POST <code>/api/v1/tasks</code>	Protected protect, authorize('admin', 'manager')	createTask Model: Task	Create new task with priority & assignee. UI: TaskBoard.jsx
GET <code>/api/v1/tasks</code>	Protected protect	getTasks Model: Task	Query tasks by project, status, priority, search. UI: TaskBoard.jsx
GET <code>/api/v1/tasks/:taskId</code>	Protected protect	getTaskById Model: Task	Get task details with assignee & creator populated. UI: TaskDetails.jsx
PUT <code>/api/v1/tasks/:taskId</code>	Protected protect (RBAC inside)	updateTask Model: Task	Update status (Dev) or full details (Admin/Mgr). UI: TaskBoard.jsx, TaskDetails.jsx
DELETE <code>/api/v1/tasks/:taskId</code>	Protected protect, authorize('admin', 'manager')	deleteTask Model: Task	Delete task and associated sub-resources. UI: TaskBoard.jsx, TaskDetails.jsx
POST <code>/api/v1/tasks/:taskId/comments</code>	Protected protect	createComment Model: Comment	Add comment to task discussion thread. UI: TaskDetails.jsx
GET <code>/api/v1/tasks/:taskId/comments</code>	Protected protect	getCommentsByTaskId Model: Comment	Fetch comments for a task. UI: TaskDetails.jsx
PUT <code>/api/v1/comments/:commentId</code>	Protected protect	updateComment Model: Comment	Edit comment (Author only). UI: TaskDetails.jsx
DELETE <code>/api/v1/comments/:commentId</code>	Protected protect	deleteComment Model: Comment	Delete comment (Author / Project Owner). UI: TaskDetails.jsx
POST <code>/api/v1/tasks/:taskId/attachments</code>	Protected protect, uploadMiddleware	uploadAttachment Model: Attachment	Upload file attachment (PDF, DOCX, PNG, JPG). UI: TaskDetails.jsx
GET <code>/api/v1/tasks/:taskId/attachments</code>	Protected protect	getAttachments Model: Attachment	Fetch all attachments linked to task. UI: TaskDetails.jsx
DELETE <code>/api/v1/attachments/:attachmentId</code>	Protected protect	deleteAttachment Model: Attachment	Delete file attachment and unlink disk file. UI: TaskDetails.jsx

GET /api/v1/dashboard/overview	Protected protect	getOverview Model: Cross-model	Aggregate KPI counts (projects, tasks, workspaces). UI: Dashboard.jsx
GET /api/v1/dashboard/task-status	Protected protect	getTaskStatus Model: Task	Task counts grouped by status (Todo, In Progress, Done). UI: Dashboard.jsx
GET /api/v1/dashboard/task-priority	Protected protect	getTaskPriority Model: Task	Task counts grouped by priority (Low, Med, High). UI: Dashboard.jsx
GET /api/v1/dashboard/recent-tasks	Protected protect	getRecentTasks Model: Task	Get 5 most recently created/updated tasks. UI: Dashboard.jsx
GET /api/v1/dashboard/recent-activities	Protected protect	getRecentActivities Model: ActivityLog	Get recent activity audit log entries. UI: Dashboard.jsx
GET /api/v1/dashboard/my-tasks	Protected protect	getMyTasks Model: Task	Get tasks assigned to logged-in user. UI: Dashboard.jsx
GET /api/v1/notifications	Protected protect	getNotifications Model: Notification	Get notification history for logged-in user. UI: NotificationDropdown.jsx
PATCH /api/v1/notifications/read-all	Protected protect	markAllAsRead Model: Notification	Mark all notifications as read for current user. UI: NotificationDropdown.jsx
PATCH /api/v1/notifications/:id/read	Protected protect	markAsRead Model: Notification	Mark single notification as read. UI: NotificationDropdown.jsx

8. Complete Project Workflow & End-to-End Journey

The following sequential walkthrough illustrates how DevFlow coordinates multi-role user journeys from initial enrollment through real-time task execution:

Step 1: Account Creation & Direct Onboarding

User accesses `/register`, submits credentials. Backend validates format, checks duplicate emails, hashes password via bcrypt, and saves User document. Frontend displays confirmation and redirects to `/login`.

Step 2: Authentication & Token Provisioning

User enters credentials on `/login`. Backend validates password hash, signs JWT Access Token (15m) and Refresh Token (7d). Client stores access token and user payload in localStorage and redirects to `/dashboard`.

Step 3: Executive Dashboard & Workspace Context

User lands on `/dashboard`. System fetches active workspaces and aggregates team metrics. User selects target workspace, synchronizing workspace context in the global navigation bar.

Step 4: Project Provisioning (Admin & Manager)

Admin or Manager opens `/projects`, clicks 'Create Project', inputs project charter. Server verifies `authorize('admin', 'manager')`, records audit log entry, and associates project with workspace.

Step 5: Task Authoring & Assignment Delegation

On `/task-board`, Manager creates task 'Build Auth Interceptor' and assigns it to 'Divyanshu (Developer)'. Backend persists task, creates `TASK_ASSIGNED` notification in MongoDB, and emits live WebSocket event.

Step 6: Real-Time Notification Receipt

Developer's browser receives Socket.IO `notification:new` event. Ephemeral Toast banner pops up on screen and Navbar bell counter increments from 0 to 1 without refreshing the page.

Step 7: Task Execution & Discussion Collaboration

Developer clicks notification or task card, navigating to `/tasks/:id`. Posts comment 'Investigating token expiry behavior'. Author can edit or delete comments; collaborators receive `COMMENT_ADDED` alerts.

Step 8: Asset Attachment & File Sharing

Developer uploads `architecture-diagram.pdf` via the attachment panel. Multer validates extension and size, saves file to server `uploads/`, and stores metadata in `attachments` collection.

Step 9: Kanban State Transition

Developer drags or selects status dropdown, updating task to 'In Progress' or 'Done'. Server verifies role permission, updates document, logs activity event, and notifies project manager.

Step 10: Audit Log Verification & Sprint Review

Team opens `/activity` to review immutable timeline of all sprint actions, ensuring end-to-end accountability across the project lifecycle.

9. Security Architecture & Threat Mitigation

Security Threat / Vector	DevFlow Defense Mechanism	Implementation Details
Credential Theft & Brute Force	Bcrypt Password Hashing with Salt Rounds	Passwords hashed with 10 salt rounds in Mongoose pre-save hook. Passwords never returned in queries (<code>select: false</code>).
Unauthorized Endpoint Access	Stateless JWT Token Verification Middleware	<code>protect`</code> middleware verifies HMAC SHA-256 signature, validates expiration, and verifies user <code>isActive`</code> status in DB.
Privilege Escalation	Granular Role-Based Access Control (RBAC)	<code>authorize('admin')`</code> and <code>authorize('admin', 'manager')`</code> guard sensitive mutations before controller logic executes.
NoSQL Injection & Parameter Tampering	Mongoose Schema Validation & Escaped Regex	Strict schema typing blocks unknown keys. Search queries escape regex meta-characters (<code>replace(/[.*+?^\${}() [\]]/g, '\\\$&')</code>).
Cross-Site Scripting (XSS) & Clickjacking	Helmet Security Headers & React JSX Escaping	Helmet applies X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, and Content-Security-Policy headers.
Arbitrary File Upload Exploit	Multer Extension Whitelist & File Size Caps	Strict whitelist (<code>.pdf`, `.docx`, `.png`, `.jpg`, `.jpeg`</code>) with 5MB ceiling prevents executable file execution.
WebSocket Impersonation	Authenticated Socket Handshake & Scoped Rooms	Socket.IO verifies JWT during connection handshake. Clients only join their own private room (<code>socket.join(user._id)`</code>).

10. Architectural Differentiators & USPs

- **Dual-Channel Synchronization:** Merges standard REST API transactions with instant WebSocket push updates, eliminating noisy polling cycles.
- **Strict 3-Tier Hierarchy:** Enforces clear workspace-to-project-to-task containment, ensuring multi-team boundaries without database bloat.
- **Zero-Redux Lean State Architecture:** Uses specialized React Context providers with local storage synchronization, resulting in faster load times and minimal memory footprint.
- **Built-in Enterprise Auditability:** Every project and task mutation writes immutable ActivityLog records, providing native compliance tracking.

11. Scalability & Engineering Roadmap

Enhancement Area	Current State	Planned Enterprise Evolution
File Storage	Local server disk storage in <code>uploads/`</code>	Migrate to AWS S3 / Cloudflare R2 with pre-signed URLs and CDN edge caching.

Caching Layer	Direct MongoDB queries with indexes	Integrate Redis caching for dashboard aggregates, user profiles, and active projects.
AI Assistance	Manual task specification authoring	Integrate Gemini / Claude API for automated task breakdowns and sprint planning.
Mobile Experience	Responsive web viewport (Tailwind)	Develop cross-platform React Native mobile client reusing REST & Socket endpoints.

12. Technical Interview Preparation Masterclass

12.1 The 2-Minute Executive Pitch

'DevFlow is a full-stack project collaboration and task management platform built with React 19, Node.js, Express, and MongoDB Atlas. It solves multi-tier workflow coordination through a structured hierarchy of Workspaces, Projects, and Kanban Task Boards. What distinguishes DevFlow is its enterprise Role-Based Access Control dividing privileges between Admins, Managers, and Developers, coupled with real-time Socket.IO event push for instant notifications. It features rich collaboration via threaded comments, secure 5MB multipart file attachments, immutable audit logging, and dynamic dashboard analytics. In short, it brings the agility of Jira and Linear into a streamlined, high-performance architecture.'

12.2 The 5-Minute Technical Deep-Dive

'Architecturally, DevFlow is designed with clean separation of concerns. On the frontend, we use React 19 with Vite and Tailwind CSS. Instead of introducing heavy state management libraries like Redux, we designed three modular Context providers: AuthContext for token and session lifecycle, NotificationContext for live WebSocket management, and ThemeContext. An Axios request interceptor injects the Bearer JWT token into every outbound call.

On the backend, Express 5 mounts modular routers under `/api/v1` guarded by a two-stage middleware pipeline: `protect` verifies token signatures and active account status, while `authorize` enforces strict RBAC rules. The database schema in Mongoose leverages normalized references with automatic population. For instance, tasks populate assignee, project, and creator details. When state changes occur, our controllers perform two critical actions: they record an immutable ActivityLog entry and broadcast a WebSocket event to the recipient's private room. For file uploads, Multer enforces disk storage and strict extension whitelisting. The codebase is fully modular, adhering to single-responsibility principles across all 44 endpoints.'

12.3 Key Engineering Challenges & Solutions Implemented

- **Challenge 1: Real-Time Notifications Without Polling Overhead:** Solution: Integrated Socket.IO directly onto the Express HTTP server in `server.js`. Authenticated clients join private user rooms (`socket.join(user._id)`). Controllers trigger `emitNotification`, pushing alerts immediately to the active client without polling.
- **Challenge 2: Multi-Role Task Permission Nuances:** Solution: Implemented dual-layer authorization. Route-level `authorize('admin', 'manager')` prevents developers from creating/deleting tasks, while controller-level inspection permits assigned developers to update task status while strictly forbidding unauthorized reassignments or metadata changes.
- **Challenge 3: Token Synchronization & Automatic Session Recovery:** Solution: Configured centralized Axios interceptors coupled with `AuthContext` initialization logic that queries `GET /auth/me` on application boot, automatically recovering valid sessions or purging expired tokens cleanly.
- **Challenge 4: File Upload Security & Integrity:** Solution: Configured Multer disk storage with unique timestamped UUID suffixes, enforced a 5MB ceiling, and implemented strict extension whitelisting (`.pdf`, `.docx`, `.png`, `.jpg`, `.jpeg`) to eliminate arbitrary script execution.